Use LLMs to Build Repair Tools, Not Patch Generation: Cost-Effective Automated Program Repair

Anonymous submission

Abstract

Automated program repair can give every student in a programming course immediate feedback. Reference-guided repair transplants from the correct programs a class has already submitted and costs nothing per submission, but fails where the pool holds no structurally matching program. LLM-based repair has no such limit, yet it issues one query per submission, so spending grows with enrollment and recurs every term while the queries to one assignment re-read the same specification and re-derive the same few misconceptions hundreds of times. We present CoEP, in which the model writes the repair *tool* rather than the patch. Once per assignment, a builder LLM turns worked pairs drawn from the class's own submissions into a module of misconception-named rewrite rules over a reference-guided transplant, validated against the instructor's test cases and then frozen. Submissions are repaired by that module with no model at all, and only what it abstains on, or what it patches more heavily than the program is long, takes a conventional call; the repairs it cannot make are distilled back into rules it keeps, so the artifact develops with the assignment it serves. What is deployed is not a model but code that can be read, audited, and that names the misconception behind every repair.

Introduction

Every year, thousands of students enroll in programming courses, and online judges accumulate millions of submissions to the same set of problems. With the rapid growth of Computer Science enrollment, providing personalized and timely feedback on programming assignments has become a significant challenge, requiring substantial time and effort from faculty (Yi et al. 2017; Hu et al. 2020).

Automated Program Repair (APR) has emerged as a promising solution to this challenge, aiming to deliver automated and personalized feedback to students about their programming errors (Gulwani, Radiček, and Zuleger 2018; Wang, Singh, and Su 2018; Hu et al. 2020; Ahmed et al. 2022). Reference-guided APR aligns a buggy program with a correct program for the same problem and transplants the differing region. It uses no model at deployment, so it runs offline and deterministically at no marginal cost per submission. Its applicability depends on the pool of correct programs containing a structurally matching correct program: when the alignment is not possible, the tool reports a structural mismatch and the program is returned unrepaired (Orvalho, Janota, and Manquinho 2025).

More recently, Large Language Models (LLMs) trained on code have shown great potential in repairing programs (Xia and Zhang 2022, 2024; Zhang et al. 2024; Liu et al. 2024; Yang et al. 2024). LLM-based APR does not require a matching reference, so it applies where a structural mismatch would otherwise arise, and it extends beyond introductory exercises to harder problems. What concerns us is how it is deployed. The default is one model query per buggy program, so spending, latency, and dependence on an external service all scale with the number of submissions and recur every term. Queries to the same assignment also share most of their content: hundreds of students attempt one problem and their failures cluster into a few recurring misconceptions (Gulwani, Radiček, and Zuleger 2018; Hu et al. 2020), so the model re-reads one specification and re-derives one mistake many times over. Cheaper tiers and prompt caching lower the price of a call (Anthropic 2025; OpenAI 2024) but leave the number of calls unchanged.

In this paper, we propose CoEP (Cost-Effective Program repair), in which the model writes the repair *tool* rather than the patch generation. The model is invoked once per problem instead of once per submission, and the product of that invocation is not a single repair but an executable program that handles every submission to the problem. An LLM agent receives the problem description, its test cases, and a clustered sample of (buggy, fixed) program pairs drawn from the class, and authors AST rewrite rules named after the misconceptions they repair. These rules run atop a shared symbolic substrate of a generic edit library and reference-guided minimal-hunk transplant from the pool of correct programs. Figure 1 shows one such rule. On cAPS lOCK, 108 of 560 buggy programs test a word's tail with `a[1:].isupper()`, which is `False` on the empty tail of an input like `"Z"`. Repairing them one submission at a time re-derives the same fix 108 times, and a transplant has to bring in a length guard from the pool together with the structure around it. One rule instead rewrites that test in place, repairs all 108, of which the builder was shown 64 and never saw the other 44, at no

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

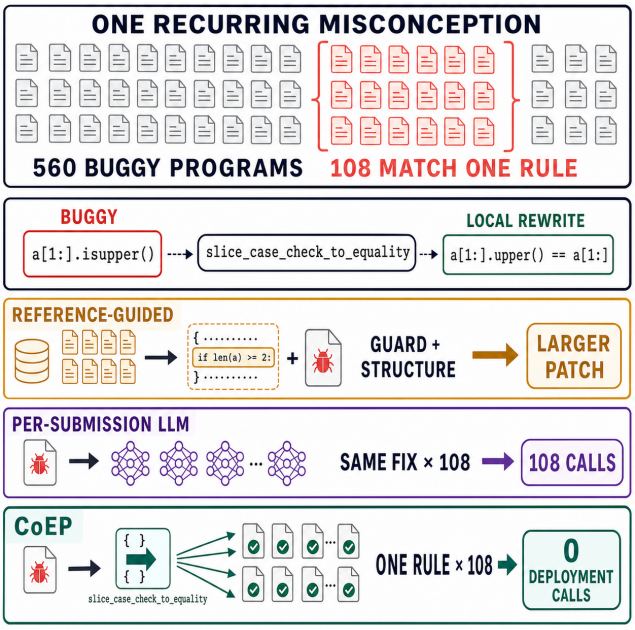


Figure 1: Example of CoEP: A recurring misconception on cAPS lOCK and the rule that repairs it.

deployment cost. Every candidate fixed program is validated against the instructor's test cases before it is kept, so the tool can never self-report success, and a defective tool degrades into abstention rather than into a wrong repair. Only the few submissions the tool abstains on, together with those whose local patch is larger than the program it repairs, escalate to a conventional per-submission call, so coverage is never sacrificed. Model spend therefore moves from a marginal cost that scales with submissions to a fixed cost that scales with problems, and the deployed system retains no model: what a student's submission meets is code that can be read and audited, and that runs offline inside the institution.

Our experiments with 2,970 buggy programs across 11 problems from two corpora, one of course assignments and one of online judge problems, reveal that the synthesized tool outperforms Refactory, the strongest reference-guided repairer that runs on our data, on its own corpus and metric, repairing more programs with smaller patches. Against per-submission LLM repair it matches coverage while modifying less of the student's program, for a fraction of the model spend, and it recovers its one-off build cost within tens of submissions per problem. An ablation separates what each layer contributes. On a benchmark whose assignments already carry hundreds of correct submissions, most of the reach comes from the symbolic substrate and the authored rules add smaller patches and a labelled account of which misconception produced each repair; restrict the pool the transplant may draw on and the balance inverts: with none of it the substrate repairs a fraction of what the same tool with its rules repairs. Those two are what a per-submission call cannot amortize, since a call yields a patch but leaves no reusable artifact behind.

In summary, this paper makes the following contributions:

- We reformulate APR for programming assignments so that the model authors a repair tool once per assignment rather than a patch per submission, leaving no model in the deployed system;
- We present CoEP, which synthesizes misconception-named rewrite rules atop a reference-guided transplant, and keeps developing the tool by distilling the repairs it could not make into new rules;
- Experiments on real student submissions show that CoEP repairs more programs with smaller patches than a reference-guided tool, and matches per-submission LLM repair at an order of magnitude lower cost.

Related Work

APR for programming assignments. Early work reuses the correct programs a class has already submitted. CLARA and Sarfgen align a buggy program with its structurally nearest correct program and transplant the differing blocks (Gulwani, Radiček, and Zuleger 2018; Wang, Singh, and Su 2018), Verifix backs that alignment with formal verification (Ahmed et al. 2022), and Refactory widens the range of alignable programs by refactoring the pool to match the buggy program's structure (Hu et al. 2020). A second line learns the repair function from data, with sequence models for syntactic errors (Gupta et al. 2017), reinforcement learning over edit actions (Gupta, Kanade, and Shevade 2019), and PROSE-based induction of per-assignment transformations from (buggy, fixed) program pairs (Rolim et al. 2017). Most recently, LLMs trained on code address the problem directly: PyDex targets student programs and prepends similar correct programs retrieved by test-case behaviour as few-shot examples (Zhang et al. 2024), ChatRepair adds multi-turn conversation (Xia and Zhang 2024), CREF places a tutor in the loop (Yang et al. 2024), FastFixer fine-tunes for faster per-query inference (Liu et al. 2024), and recent work extends to counterexample-guided minimal repair (Orvalho, Janota, and Manquinho 2025) and learner-tailored retrieval (Dai et al. 2026). Alongside repair itself, OverCode clusters a class's programs to show an instructor which mistakes recurred (Glassman et al. 2015), the same instructor-facing view our rule provenance yields as a by-product of repairing. **Letting a model build the tool.** The reformulation this paper applies to repair has been made elsewhere. Cai et al. (2024) have a capable model write a reusable utility once and a cheaper model call it many times, amortizing the expensive model over a task family. Voyager accumulates a skill library from experience (Wang et al. 2024), TroVE and ReGAL induce reusable functions from programmatic tasks (Wang, Fried, and Neubig 2024; Stengel-Eskin, Prasad, and Bansal 2024), and FunSearch evolves programs against a scoring function (Romera-Paredes et al. 2024). What repair contributes to this line is an oracle. A tool induced for reasoning tasks is trusted because it looks right, whereas every patch ours emits is executed against the instructor's test cases before a student sees it, so a defective rule degrades into abstention rather than into a wrong answer, and the tool can be shipped without a model behind it. The by-product differs

too: because each rule is written for a named misconception, repairing a class also labels it.

Three of these are close enough to ours that their absence from our baselines needs stating. Refazer is the closest in spirit, since it too induces assignment-specific transformations, but it learns them from a student's own (before, after) edit, and neither corpus we use records which correct submission a given buggy one became; its input does not exist in our data. Verifix and the counterexample-guided repair of Orvalho, Janota, and Manquinho (2025) both operate on C, the first through control-flow alignment with MaxSMT over C types and the second through a C-specific MaxSAT fault localizer, while both of our corpora are Python. We therefore compare against Refactory, the strongest reference-guided tool that runs on our data, and PyDex, which represents the per-submission LLM line. Existing approaches, reference-guided and LLM-based alike, either fix the repair procedure by hand in advance or re-derive it from the model on every submission; neither leaves behind a reusable artifact for the assignment. To remedy this gap, we have the model author that artifact once per assignment, and repair every later submission with it and no model at all.

Approach

Task Definition

A programming assignment provides a problem description d, a set of test cases $T = \{t_1, \ldots, t_n\}$, and a set of submissions. Writing $p \models T$ when a program p passes every test case in T, the submissions divide into the buggy programs $B = \{p : p \not\models T\}$ and the correct programs $R = \{p : p \models T\}$. Since the correct program used to repair a buggy program is a reference program, R is also the pool of reference programs. Repairing the buggy programs produces a set of fixed programs F, with individual members denoted b, r, and f. To repair b is to produce an f with $f \models T$, and nothing weaker counts: an f passing only some test cases is not a repair.

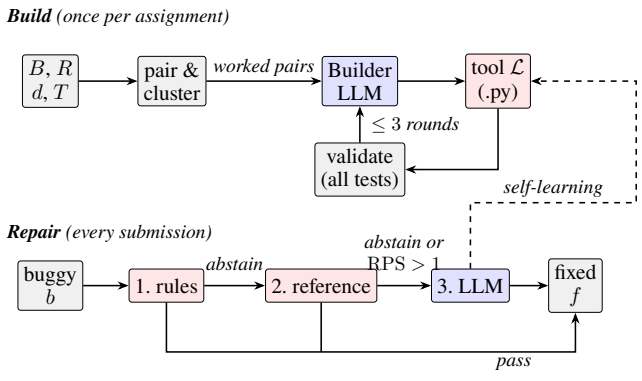


Figure 2: Overview of CoEP. LLM effort (blue) is spent once per assignment to build a local repair tool (red); at repair time the tool tries rules and a reference transplant first, so most submissions never reach a model. The dashed path is self-learning ($\text{CoEP}_{\text{L+SL}}$).

Overview

CoEP separates the two things a repair system does (Figure 2). The *build* happens once per assignment. A local step that costs no model call turns the class's own submissions into a small set of worked pairs, and a builder LLM turns those, together with d and T, into a *local repair tool* $\mathcal{L}$: an ordinary Python module with no model access. *Repair* happens on every submission and consults $\mathcal{L}$ first. Given a buggy program b, the tool tries three strategies in increasing order of cost: its own rewrite rules, a transplant from a reference program, and, only if both abstain or the local patch is larger than the program it repairs, a single conventional model call. Whichever strategy first produces a candidate that passes every test case in T supplies the fixed program f, so a model is involved in the third alone.

Three configurations follow from where we stop. CoEP_{L} uses the tool alone and spends nothing per submission beyond CPU time. CoEP_{LF} adds the model call for the submissions the tool cannot serve. $\text{CoEP}_{\text{L+SL}}$ adds self-learning: whenever the fallback repairs a submission the tool missed, the model generalizes that repair into a new named rule and the tool absorbs it, so an assignment's tool improves over a semester rather than staying frozen at whatever one build session produced.

Build: Local Repair Tool

Worked pairs. This step is local and costs no model call: the builder sees a *chosen* sample of B rather than all of it. Let $\sigma(p)$ collect the node types of p's AST read in breadth-first order, one sequence $\sigma_i(p)$ per top-level statement, and, with lev the Levenshtein distance between two such sequences, let $\delta(b, r)$ be the cost of the cheapest one-to-one matching M of b's sequences to r's, $\min_M [\sum_{(i,j)\in M} \mathrm{lev}(\sigma_i(b), \sigma_j(r)) + \sum_{i \notin M} |\sigma_i(b)| + \sum_{j \notin M} |\sigma_j(r)|]$, so a missing or surplus definition costs what it is. Every buggy program takes as partner the reference program it is fewest edits from, $\pi(b) = \arg\min_{r \in R} \delta(b, r)$, rather than the one that merely looks closest, and $\varepsilon(b)$ is the multiset of edit operations taking b to $\pi(b)$, refined to a value-level alignment when the node-type one comes out empty, since a wrong identifier or literal is invisible to node types alone.

We then partition $B = C_1 \uplus \cdots \uplus C_k$ by average linkage over cosine distance between TF-IDF vectors of the $\varepsilon(b)$, with k chosen to maximize mean silhouette. Both choices were measured rather than assumed. Clustering on $\varepsilon(b)$ and not on the program's own AST-path n-grams, because a rule induced from a representative pays off on the members needing the *same edit*, whereas grouping by shape reliably fills the largest clusters with programs that merely look alike; and k from the data, because a fixed cut height is not a smaller assumption than a fixed k and one constant produced six-fold different granularity across our assignments. Each cluster contributes its *modal* repair, $b_i^\star = \arg\max_{b \in C_i} |\{b' \in C_i : \varepsilon(b') = \varepsilon(b)\}|$, the member whose edit the cluster most agrees on. The two obvious alternatives survive only as the fallback for a cluster whose edits never repeat, where $b_i^\star$ degenerates to the medoid: the medoid is the most typical *program* and is usually a multi-

issue tangle, while the smallest edit is the submission that came closest to correct, an outlier rather than a spokesman. The build set is $S = \{(b_i^\star, \pi(b_i^\star))\}_{i=1}^{k}$, ordered by increasing δ, so a weak builder is shown each buggy program *with* a reference program that repairs it and need only generalize the transformation.

Rule authoring. The builder LLM turns (d, T, S), with T shown as up to eight example cases, into the assignment-specific tool $\mathcal{L}_0$: a module of AST rewrite rules, each named after the misconception it targets and tagged with the failure symptoms it addresses. A rule is a *candidate generator*, a map ρ from a program to a set of candidate programs, and the contract is strict. It locates its edit on the AST and splices it as a source-span replacement, never re-rendering the module, which would reformat untouched lines. It proposes rather than claims, and it never rewrites a whole function: a rule that cannot express a targeted edit, that raises, or that runs past a two-second alarm yields $\rho(p) = \emptyset$ and the submission escalates.

Generate–validate–refine. A single-shot module can contain its own defects, so the harness closes the loop on the build set itself, in the manner of self-refinement from execution feedback (Madaan et al. 2023; Shinn et al. 2023). Write $\mathrm{cov}(\mathcal{L})$ for the fraction of S whose buggy half the rule search below repairs, $|\{(b, \cdot) \in S : \exists\, c \in \mathcal{L}^*(b),\ c \models T\}|/|S|$, with $\mathcal{L}^*(b)$ the candidate set that search enumerates; the transplant sits deliberately outside it, so a round is scored on the rules the builder wrote and not on the substrate underneath them. Coverage is measured by *executing* T, never from the module's own claim. While $\mathrm{cov}(\mathcal{L}_j) < 1$ the harness returns the still-failing sources for revision, giving $\mathcal{L}_{j+1}$, for up to $r_{\max}{=}3$ builder calls; the frozen tool is $\mathcal{L} = \arg\max_{j \le r_{\max}} \mathrm{cov}(\mathcal{L}_j)$, ties going to the earliest round. Build cost is exactly the token count of these exchanges, measured from the raw strings.

Repair: Rules, Reference, LLM

Given b, the frozen tool $\mathcal{L}$ tries three strategies in increasing order of cost and returns the first candidate c it can verify with $c \models T$. The first two use no model; only the third spends a call.

Rule-based repair. The rules are composed by a symptom-ordered search over layers: $\Lambda_0 = \{b\}$ and $\Lambda_{j+1} = (\bigcup_{p \in \Lambda_j} \bigcup_{\rho \in \mathcal{L}} \rho(p)) \setminus \bigcup_{i \le j} \Lambda_i$, with b's failing-test signature ordering the ρ whose declared symptoms match. Every candidate in Λ_{j+1} is validated before Λ_{j+2} is enumerated, so the returned repair is by construction the shallowest c with $c \models T$, and $\mathcal{L}^*(b) = \bigcup_j \Lambda_j$. A handful of always-on rules for the slips every class makes, comparison flips and off-by-one range and index shifts, sit alongside whatever the builder writes; they invert the classical mutation-operator space (Just et al. 2014; Kim et al. 2013; Liu et al. 2019). Every surviving patch carries the ordered rule names behind it, provenance that aggregates into a per-class *misconception profile*.

Reference-based repair. Most student bugs need a structural edit no token-level rule expresses; R supplies it. Let $H(b, r)$ be the differing line-hunks between b and one of the eight structurally nearest reference programs r, taken both raw and with r's variables renamed to the student's, parameters by position and locals by first binding, and let $b \oplus h$ be b with the hunks $h \subseteq H(b, r)$ applied. We validate the 300 smallest h with $1 \le |h| \le 3$ in increasing order of lines changed and return $\arg\min_h \mathrm{RPS}(b, b \oplus h)$ over the first five satisfying $b \oplus h \models T$, abstaining when none does, so the exact tree-edit score, the distance from b to $b \oplus h$ normalized by the size of b and defined with the other metrics in the Experimental Setup, is paid only on the handful that pass. This is reference reuse in the lineage of CLARA, Sarfgen, and Refactory (Gulwani, Radiček, and Zuleger 2018; Wang, Singh, and Su 2018; Hu et al. 2020), at hunk granularity rather than the block granularity those systems use.

LLM-based repair. Conventional LLM-based APR maps every buggy program to a query $\mathrm{LLM}(d, T, b)$. CoEP keeps that call for the submissions the local strategies cannot serve, and its cost is counted against CoEP in every comparison. Writing f_{loc} for the local result, it fires when $f_{\mathrm{loc}} = \mathrm{fail}$ or $\mathrm{RPS}(b, f_{\mathrm{loc}}) > 1$. Abstention is not the only trigger because the acceptance test is the instructor's suite, which says nothing about patch size: the transplant can return a $c \models T$ larger than the student's whole program, while the rule layer must return $\emptyset$ whenever it cannot express a targeted edit. The threshold gives the transplant the same contract, since a patch larger than the program it repairs is a rewrite, not a repair, under any assignment's standards. The call is strengthened to match the student's strategy by attaching the minimum-edit reference program $r = \pi(b)$, the same pairing the build uses, and we keep $\arg\min_{f \in \{f_{\mathrm{loc}},\, \mathrm{LLM}(d,T,b,r)\},\ f \models T} \mathrm{RPS}(b, f)$, abstaining when neither passes, so escalating can cost a call but never coverage or patch size.

The threshold is a constant rather than a quantity calibrated per assignment. We chose it that way because its value is a definition and not a fitted number; the supplementary material reports a sweep over 29 measured settings, across which the repair rate does not move.

Self-learning. From an escalated pair (b, f) the LLM writes a new named rule ρ', kept only if $\exists\, c \in (\mathcal{L} \cup \{\rho'\})^*(b)$ with $c \models T$, in which case $\mathcal{L} \leftarrow \mathcal{L} \cup \{\rho'\}$. Absorption writes to a staging copy the evaluated pass never consults, so no submission's outcome depends on which preceded it, and the accumulated rules are a separate artifact, evaluated by a second frozen pass and priced to themselves. This is the $\mathrm{CoEP}_{\mathrm{L+SL}}$ configuration; it spends calls only on submissions that already escalated, so it too leaves the build claim untouched.

The first two strategies run on a CPU in ∼2–3 s per submission and offline. Under $\mathrm{CoEP}_{\mathrm{L}}$ no student code leaves the institution once the tool is built; the build itself sends the worked pairs to a provider, and $\mathrm{CoEP}_{\mathrm{LF}}$ sends each escalated submission, so the guarantee is a property of the configuration and not of the method. Rule application and transplant are themselves deterministic; the enumeration that feeds them is bounded by a wall clock, so under heavy load fewer candidates are examined and the tool may abstain where it would otherwise repair. A defective tool fails, never returning a wrong repair, because the harness re-validates every patch before a student sees it.

Experiments

Research Questions.

- **RQ1**: What is the cost impact of synthesizing a repair tool once per assignment, compared to querying an LLM once per submission?
- **RQ2**: How effective is CoEP in repairing programming assignments compared to state-of-the-art reference-guided and LLM-based repair approaches?
- **RQ3**: How does the LLM-authored rule layer impact the performance of CoEP?
- **RQ4**: How helpful is it to absorb the repairs of escalated submissions back into the tool as new rules?

Baselines. We compare CoEP against the two mainstream lines of program repair for programming assignments: (1) **Refactory** (Hu et al. 2020), which refactors the class's correct programs to match each buggy program's structure and transplants the divergent blocks; (2) **PyDex** (Zhang et al. 2024), which sends each buggy program to the model in its few-shot layout with the three most similar (buggy, correct) exemplars retrieved from the class pool, samples $K{=}10$ candidates per submission, filters them by the test oracle, and returns the survivor closest to the student's program; every one of the ten calls is billed and timed, which is what a per-submission arm costs by construction. (3) **PyDex$_{\text{PC}}$** re-prices the same queries under *prompt caching*. Providers let a query reuse the key–value tensors computed for an identical leading prefix of an earlier one, so the cached prefix is billed at roughly a tenth of the input rate and need not be re-encoded (Anthropic 2025; OpenAI 2024). CoEP itself is evaluated in three configurations. **CoEP$_{\text{L}}$** runs the frozen local tool alone with no deployment LLM use, accepting any patch the test oracle passes, which is the acceptance rule Refactory uses. **CoEP$_{\text{LF}}$** adds the per-submission fallback on abstentions and on passing local patches above $\text{RPS} = 1$. **CoEP$_{\text{fw}}$** is the framework alone, the same tool with the builder's rules removed and the always-on rules and the transplant left in place, which calls no model at any point. **CoEP$_{\text{L+SL}}$** re-runs CoEP$_{\text{L}}$ with the self-learned rules merged and frozen beforehand, so the arm is order-independent; its induction calls are priced to it.

Corpus	problems	buggy	correct	tests
Refactory (CS1)	5	1,783	2,442	37
Codeforces	6	1,187	698	213
Total	**11**	**2,970**	**3,140**	**250**

Table 1: Datasets Statistics.

Datasets. We evaluate on 11 problems and 2,970 buggy programs drawn from two real-student corpora (Table 1): five CS1 assignments contributing 1,783 buggy and 2,442 correct programs at 7.4 test cases per assignment, and six Codeforces problems contributing 1,187 buggy and 698 correct at 35.5 test cases. The first five are the assignments of a first-year university programming course, with every submission its students actually wrote, released by the authors of Refactory (Hu et al. 2020) and since adopted as the standard corpus of this literature (Zhang et al. 2024). The other six are problems from Codeforces, one of the largest public online judges, whose submissions the open-r1 release mirrors (Penedo et al. 2025); we sample them by the difficulty rating, one problem per band from 800 to 2000, covering five official rating bands from Newbie to Candidate Master (Codeforces 2024).

LLM Configuration. Every LLM arm runs on two models, one per provider: **gpt-5.4-mini** ($0.75/$4.50 per Mtok) and **claude-sonnet-5** ($2/$10 per Mtok). We select each provider's cheapest tier at which prompt caching is functional, in fairness to the caching baseline: PyDex$_{\text{PC}}$ only competes if caching can engage, and we verified that the absolute cheapest tiers rule it out: Haiku 4.5 refuses to cache below its 4,096-token minimum, above these prompts, and GPT-5.4-nano's automatic caching did not engage on identical 1,200-token prompts, whereas both selected models cache at their documented 1,024-token minimums.

Evaluation Metrics. **Repair rate (RR)** is the percentage of buggy programs a system repairs. A repair counts only when the fixed program passes *all* of the assignment's test cases; a fixed program that passes some of them is not a repair. **Relative patch size (RPS)** is the tree-edit distance between the buggy and fixed ASTs normalized by buggy program size, reimplemented from Refactory's definition (Hu et al. 2020), and is averaged over what an arm repairs. **Average time taken** $\text{ATT}_{\text{tot}} = \text{ATT}_{\text{cpu}} + \text{ATT}_{\text{api}}$ is mean wall-clock seconds per submission, local search plus model latency. **LLM Cost** $C_{\text{tot}} = C_{\text{build}} + C_{\text{api}}$ is US dollars over the whole benchmark, the one-off synthesis plus the per-submission inference, priced at the tiers above from exact token counts; a system with no build phase has $C_{\text{build}} = 0$. The **escalation rate** e is the fraction of submissions taking a model call.

Results

RQ1: Cost Effectiveness. Our objection to per-submission repair was structural: spending scales with enrollment and recurs every term, and a cheaper tier or a cache lowers the price of a call without changing how many calls are made. This question answers what moving the call to once per assignment costs, how many submissions repay the one-off build, and what the arrangement costs in turnaround. Figure 3 plots the cumulative C_{tot} one assignment spends against the submissions it receives, once per model: a per-submission arm is a line through the origin whose slope is the price of one query, while CoEP starts at C_{build} and rises only on the fraction e that escalates, so with Δ the marginal per-submission saving the two cross at $N^* = C_{\text{build}}/(\Delta(1-e))$. Table 2 splits turnaround into local search and the model wait behind it.

Three findings follow. **(1) In dollars**, CoEP$_{\text{LF}}$ spends $C_{\text{tot}} = \$13.58$ over the whole benchmark on claude-sonnet-5 and $1.00 on gpt-5.4-mini, against PyDex's $292.77 and $28.42, **21.6 and 28.4 times** ours, a ratio that scales with the $K{=}10$ candidates PyDex draws per submission and would be near $2\times$ at $K{=}1$; prompt caching leaves PyDex$_{\text{PC}}$ at $263.05 and $23.50, **still 19.4 and 23.5 times**, because caching pays for a shared leading *prefix* and the variable buggy program

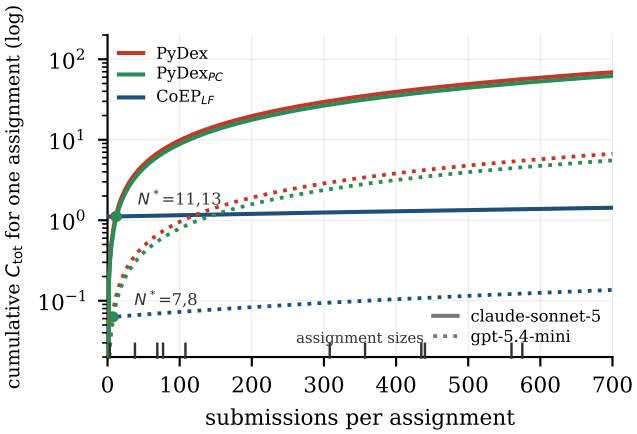


Figure 3: Cumulative C_{tot} for one assignment against its enrollment, solid for claude-sonnet-5 and dotted for gpt-5.4-mini. Dots mark where CoEP_{LF} overtakes each baseline; ticks along the axis are the eleven assignments' actual sizes, ten of which clear N^*.

Arm	Builder	ATT_{cpu}	ATT_{api}	ATT_{tot}
PyDex	`claude-sonnet-5`	0.66	13.97	14.64
CoEP_{LF}	`claude-sonnet-5`	2.70	0.36	3.06
PyDex	`gpt-5.4-mini`	0.73	1.41	2.14
CoEP_{LF}	`gpt-5.4-mini`	3.07	0.15	3.22

Table 2: Turnaround per submission, in seconds. Nearly all of PyDex's wait is the model and nearly all of ours is local search, which is why the ordering depends on how fast the model answers.

and its exemplars come first, leaving an identical prefix of about ten tokens. **(2) In break-even**, an assignment repays its build after $N^* = \mathbf{11}$ submissions against PyDex and 13 against PyDex_{PC}, on claude-sonnet-5, and after 7 and 8 on gpt-5.4-mini, where the build falls faster (17.8×) than the price of a query (10.3×) and break-even therefore arrives sooner. Ten of the eleven assignments are past either point, most by an order of magnitude. **(3) In turnaround**, ATT_{tot} is 3.06 s against 14.64 s on claude-sonnet-5 but 3.22 s against 2.14 s on gpt-5.4-mini, the one measure on which a per-submission arm can win, because a small model returns one completion faster than our search enumerates and re-executes candidates. PyDex_{PC} is absent from Table 2: it re-prices queries PyDex already issued rather than re-issuing them, so its turnaround is PyDex's exactly. What we pay for the first two is 22 minutes of build wall clock per assignment on claude-sonnet-5 (5.9–40.0), once and offline; below N^* a per-query repairer is the right tool, and since every arm runs at the cheapest tier where caching functions the measured gaps are a *floor*. Overall, moving the call from the submission to the assignment cuts spend by more than an order of magnitude on both models and repays itself within a dozen submissions, at a cost of seconds on the cheaper one.

RQ2: Repair Effectiveness. A repairer is judged on two things: how many submissions it fixes, and how much of the student's program it disturbs while fixing them. This question answers where a tool synthesized once per assignment stands on both, and how that varies across assignments. Table 3 reports per-assignment RR and RPS with the two builders pooled; RPS is averaged over what each arm repairs, so a low RPS beside a low repair rate means an easier subset rather than a better patch.

Three findings follow. **(1) Against the symbolic baseline**, the local tool alone repairs **99.1%** at mean RPS **0.463** where Refactory repairs 85.9% at 0.500, and its repair rate is at least Refactory's on every one of the eleven assignments. On Refactory's own corpus the same holds, 98.5% against its published 90.8% at 0.306 against 0.397, a patch 23% smaller, and all of it at zero marginal LLM cost. **(2) Against per-submission LLM repair**, coverage is level: CoEP_{L} reaches 99.1% and PyDex 98.9%, and adding the escalation tail takes CoEP_{LF} to **99.7%**, at 100% on ten of the eleven assignments. What separates the arms is patch size, **0.463** against **0.634**, 27% smaller, and 0.410 for CoEP_{LF}, 35% smaller; per assignment, CoEP_{L} leaves the smaller patch on ten of eleven and CoEP_{LF} on all eleven. **(3) The per-assignment rows** show what a corpus mean hides, and what predicts it. Refactory's rate swings from 40.3% to 100% while no arm carrying a model-written rule drops below 93.8%: grouped into bands, Refactory falls 90.8% on CS1 to 83.1% on Codeforces 800–1000 to **54.5%** on 1200–2000, while CoEP_{LF} holds 99.5/100.0/100.0. Thin pools alone do not explain that collapse, since C5 and C6 give every arm only two correct programs to work from and CoEP_{fw}, which keeps the transplant and drops the builder's rules, repairs both at 100%. What separates them is the granularity of reuse: Refactory adopts divergent blocks, so a pool that solves the problem another way leaves nothing adoptable, while we adopt line-hunks after renaming the reference program's variables to the student's, which stays applicable when the strategies differ. A low RPS is not always a virtue either: Refactory's 0.679 on Codeforces looks competitive against CoEP_{LF}'s 0.597, but it is averaged over the 78.6% it accepts, where CoEP_{LF} repairs 100%. Pooling the two builders hides nothing, finally: separately they reach 99.66% at 0.404 RPS and 99.73% at 0.417, three per cent apart in patch size for an eighteen-fold difference in build price. Overall, the synthesized tool beats the symbolic one on both coverage and patch size, matches per-submission LLM repair on coverage while disturbing a third less of the student's program, and does so on whichever builder writes it.

RQ3: Ablation Study. The previous two questions treated the tool as one object. This one opens it and asks what the model-authored rules add over the symbolic substrate beneath them. That substrate is CoEP_{fw}: the identical tool with the builder's rules removed and the always-on rules and the transplant left in place, which uses no model at any point and is therefore the same artifact under either builder. It is a first-class arm in Table 3, and Figure 4 gives the decomposition the ablation acts on.

Three findings follow. **(1) The reach comes from the**

	ID	Assignment	Rating	RR (%)					RPS				
				Refactory	PyDex	CoEP_{fw}	CoEP_{L}	CoEP_{LF}	Refactory	PyDex	CoEP_{fw}	CoEP_{L}	CoEP_{LF}
Refactory	R1	Sequential search	–	99.0	99.7	100.0	100.0	100.0	0.390	0.428	0.243	0.225	0.222
	R2	Unique dates/months	–	78.2	97.8	91.3	93.8	97.9	0.560	0.428	0.428	0.398	0.394
	R3	Duplicate elimination	–	97.4	99.8	100.0	100.0	100.0	0.340	0.423	0.284	0.280	0.280
	R4	Sorting tuples	–	88.2	99.3	100.0	100.0	100.0	0.310	0.535	0.388	0.386	0.291
	R5	Top-k elements	–	88.0	95.4	100.0	100.0	100.0	0.320	0.244	0.196	0.193	0.191
Codeforces	C1	Nearly Lucky Number	800	85.2	99.9	100.0	100.0	100.0	0.588	0.685	0.616	0.584	0.494
	C2	cAPS lOCK	1000	81.4	97.9	100.0	100.0	100.0	0.724	1.167	0.841	0.762	0.648
	C3	Planning Expedition	1200	47.8	100.0	98.6	98.6	100.0	0.847	0.705	0.640	0.638	0.585
	C4	Measuring Lengths	1400	92.1	100.0	100.0	100.0	100.0	0.605	0.643	0.531	0.571	0.525
	C5	Jumping Jack	1600	40.3	100.0	100.0	100.0	100.0	0.878	0.914	0.950	0.914	0.826
	C6	Digital Root	2000	100.0	100.0	100.0	100.0	100.0	2.248	1.975	1.777	1.652	1.652
	Overall			**85.9**	**98.9**	**98.7**	**99.1**	**99.7**	**0.500**	**0.634**	**0.492**	**0.463**	**0.410**

Table 3: Overall repair effectiveness on 11 assignments, with the two builders pooled. **RR** is the repair rate and **RPS** the relative patch size. CoEP_{fw} is the framework alone, the substrate with the builder's rules removed, and uses no model at any point. Refactory's CS1 rows are its published figures; its Codeforces rows are our run of the released tool.

symbolic substrate. CoEP_{fw}, which never calls a model, repairs **98.69%** at mean RPS **0.492**: far above Refactory's 85.93%, and level on coverage with PyDex's 98.94% at a smaller patch than its 0.634. We report this rather than bury it, because it is an unfavourable result for per-submission LLM repair and not for the reformulation. If 2,970 calls and $292.77 buy a quarter of a point and a larger patch over a substrate that costs no model spend at all, that spending is the thing we are objecting to. **(2) The authored layer buys the last stretch above it.** Adding it takes the repair rate to 99.06% and the mean patch to 0.463, but that 6% is the patch reduction, and it is diluted across the 75% the transplant handles either way. On the **1,447** arm–submission pairs where a rule fires, 992 distinct programs, the rule's patch is RPS **0.100** against the substrate's **0.213** on the same programs, **2.1 times** larger without it, and on 13 of them the substrate produces no repair at all, though the fallback and PyDex both do. Labelling thins with it: repairs carrying a builder-written name fall from 17.8% of all repairs to none, and only the always-on rules' names remain, over 10.1%. We claim legibility for those names, not correctness: nothing here validates that a rule's name is the misconception a tutor would assign. **(3) The substrate's reach is borrowed from the pool, and the rules are what remains when it thins.** Holding each tool fixed and varying only how many correct programs the transplant may draw on, over 203 sampled submissions, separates the two layers. Given the whole pool the arms are indistinguishable on coverage, 98.5% either way. Given ten correct programs, or five, or two, they remain indistinguishable on coverage while the rules keep the patch 10–13% smaller. Given one, the rules add 2.5 points and cut the patch 15%. Given none, the substrate has nothing to transplant from and repairs **7.4%**, while the same tool with its rules repairs **34.5%**. A course at the start of a term has no pool, and that is the regime this layer is for; a benchmark where every assignment arrives with hundreds of correct submissions already in hand is the regime that hides it. Whether $1.11 per assignment is worth that insurance is a departmental question, and a course sitting on a deep pool may reasonably decline. Our claim is not that the LLM produces the performance but that this is where LLM effort belongs.

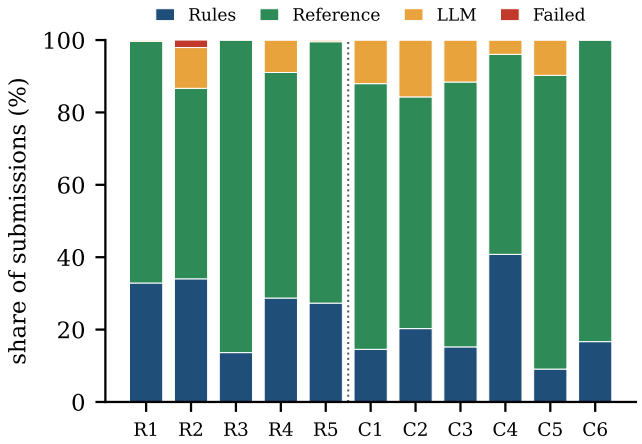


Figure 4: Decomposition of the ablation.

RQ4: Self-Learning. The frozen tool leaves a record of what it could not repair. When the fallback then repairs one of those misses, the model generalizes that fix into a named rule, and the rule is kept only if the augmented tool repairs the miss on its own. This question answers whether such a rule holds outside the submission it was induced from. Absorption writes to a staging copy the evaluated pass never consults and the augmented tool is frozen before this arm runs, so the comparison is order-independent: the same submissions, the same acceptance rule, two rule sets. Figure 5 follows what becomes of them.

Two findings follow. **(1) The material is scarce and the bar is high.** Over both builders' passes CoEP_{L} fails on **56** of 5,940 submissions, and the fallback repairs **38** of those, which are the misses induction is attempted on. Those 38

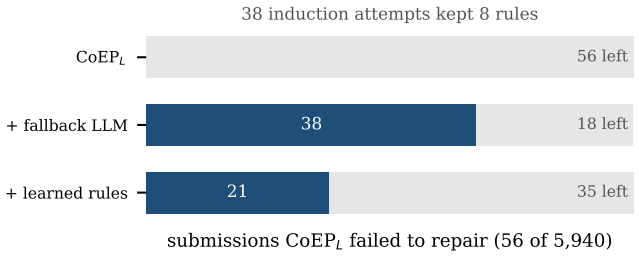


Figure 5: Self-learning, from the frozen tool's failures to the repairs its new rules produce.

attempts leave **8** rules, because a rule survives only if the augmented tool then repairs the miss on its own. That scarcity is not a failure of the mechanism but what a 99.06% local repair rate means. **(2) The rules that survive generalize past the submissions that induced them.** All eight went on to repair, recovering **21** submissions between them, and since eight submissions induced them, **13** of those 21 are programs the rules had never seen. One submission regressed, so the net is 20, and the induction cost $0.76 priced to the artifact. The cheaper builder gains more, on the same 27 abstentions on R2, which says the gain depends on how general a rule the model writes from one fix rather than on how much material it is given. Overall, self-learning fires only where the tool failed and such places are rare, but where it fires the rules carry past their own examples, for under a dollar per assignment.

Discussion

Rules come at three levels of generality. What the builder writes is not one kind of thing, and the level predicts how far a rule travels. The most general level catches a language slip. `add_missing_call_parens` describes itself as "`x.pop` / `x.reverse` used as a statement or return value without `()`", a Python trap that belongs to no assignment; it is the only one of the 128 builder-written rules that ever fired to do so on two assignments, repairing 8 submissions across R3 and C2. The middle level catches the misconception of one problem. `slice_case_check_to_equality` explains that "`slice.isupper()` fails on empty slices (single char words)" and repairs 108 submissions on C2, most of that assignment's rule-produced repairs; it cannot move to another problem but it would work for any class given this one. The most specific level is tied to the problem statement itself: `carry_fix_two_value_print` inserts an inch-to-foot carry check before the final print, fixing what it calls the "compute feet first, round remainder second" misconception, and repairs 28 of C4's 38 submissions on its own. That the authored layer is assignment-specific is therefore not a defect but a consequence of level. A rule that covers a class is written at the level of that class's misconception and so cannot cross assignments; a rule that crosses assignments does so because it knows nothing about the assignment.

A misconception profile falls out of repairing. Every rule-produced patch carries the name of the rule that produced it, so aggregating over a class yields a report an instructor can read. On R2, of the 145 submissions the rules repaired under `claude-sonnet-5`, `count_compare_relax` accounts for 58, `undefined_domain_var_rename` 27, `missing_return_false_append` 22, and `month_day_index_fix` 7, with the always-on comparison flip taking 33; a repair may compose rules, so these overlap. That is not a list of patches but a frequency table of named mistakes, and it says at once that one relaxed comparison accounts for 40% of everything the rules repaired on this assignment. The report costs nothing beyond the repairing, and a per-submission API emits none of it at any price, because a call yields a patch but leaves no name behind.

Conclusion

We reformulate program repair for programming assignments so that the model writes the repair *tool* rather than the patch: once per assignment a builder LLM synthesizes misconception-named rewrite rules over a reference-guided transplant, and the frozen tool then repairs submissions with no model at all. On 2,970 real student submissions, CoEP beats a reference-guided repairer on both coverage and patch size, matches per-submission LLM repair while editing 35% less of each program at 21.6 times lower cost, and repays its build within 11 submissions per assignment. An ablation shows that on a benchmark whose assignments arrive with deep reference pools the model buys smaller patches and a named account of each misconception rather than reach, and that the reach reappears as the model's when the pool is taken away. Since the repairs the tool cannot make are distilled back into rules it keeps, the artifact develops with the assignment it serves. As future work, we plan to test whether the same reframing holds wherever many submissions share one specification.

References

Ahmed, U. Z.; Fan, Z.; Yi, J.; Al-Bataineh, O. I.; and Roychoudhury, A. 2022. Verifix: Verified repair of programming assignments. *ACM Transactions on Software Engineering and Methodology (TOSEM)*, 31(4): 1–31.

Anthropic. 2025. Prompt Caching — Claude API Documentation. https://docs.anthropic.com/en/docs/build-with-claude/prompt-caching. Cache reads are billed at 0.1x the base input-token price; accessed July 2026.

Cai, T.; Wang, X.; Ma, T.; Chen, X.; and Zhou, D. 2024. Large Language Models as Tool Makers. In *The Twelfth International Conference on Learning Representations, ICLR 2024*.

Codeforces. 2024. Codeforces Rating System. https://codeforces.com/blog/entry/68288. Rating bands: Newbie, Pupil, Apprentice, Specialist, Expert; accessed July 2026.

Dai, Z.; Zhao, Z.; Wang, H.; Tang, X.; Wu, S.; Yao, C.; Gao, Z.; and Chen, J. 2026. Learner-Tailored Program Repair: A Solution Generator with Iterative Edit-Driven Retrieval Enhancement. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 30467–30475.

Glassman, E. L.; Scott, J.; Singh, R.; Guo, P. J.; and Miller, R. C. 2015. OverCode: Visualizing Variation in Student Solutions to Programming Problems at Scale. *ACM Transactions on Computer-Human Interaction*, 22(2): 7:1–7:35.

Gulwani, S.; Radiček, I.; and Zuleger, F. 2018. Automated clustering and program repair for introductory programming assignments. *ACM SIGPLAN Notices*, 53(4): 465–480.

Gupta, R.; Kanade, A.; and Shevade, S. 2019. Deep Reinforcement Learning for Syntactic Error Repair in Student Programs. In *Proceedings of the 33rd AAAI Conference on Artificial Intelligence, AAAI 2019*, 930–937. AAAI Press.

Gupta, R.; Pal, S.; Kanade, A.; and Shevade, S. 2017. DeepFix: Fixing Common C Language Errors by Deep Learning. In *Proceedings of the 31st AAAI Conference on Artificial Intelligence, AAAI 2017*, 1345–1351. AAAI Press.

Hu, Y.; Ahmed, U. Z.; Mechtaev, S.; Leong, B.; and Roychoudhury, A. 2020. Re-factoring based program repair applied to programming assignments. In *Proceedings-2019 34th IEEE/ACM International Conference on Automated Software Engineering, ASE 2019*, 388–398. IEEE.

Just, R.; Jalali, D.; Inozemtseva, L.; Ernst, M. D.; Holmes, R.; and Fraser, G. 2014. Are Mutants a Valid Substitute for Real Faults in Software Testing? In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering, FSE 2014*. ACM.

Kim, D.; Nam, J.; Song, J.; and Kim, S. 2013. Automatic Patch Generation Learned from Human-Written Patches. In *Proceedings of the 35th International Conference on Software Engineering, ICSE 2013*. IEEE.

Liu, F.; Liu, Z.; Zhao, Q.; Jiang, J.; Zhang, L.; Sun, Z.; Li, G.; Li, Z.; and Ma, Y. 2024. Fastfixer: An efficient and effective approach for repairing programming assignments. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, 669–680.

Liu, K.; Koyuncu, A.; Kim, D.; and Bissyandé, T. F. 2019. TBar: Revisiting Template-Based Automated Program Repair. In *Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2019*, 31–42. ACM.

Madaan, A.; Tandon, N.; Gupta, P.; Hallinan, S.; Gao, L.; Wiegreffe, S.; Alon, U.; Dziri, N.; Prabhumoye, S.; Yang, Y.; Gupta, S.; Majumder, B. P.; Hermann, K.; Welleck, S.; Yazdanbakhsh, A.; and Clark, P. 2023. Self-Refine: Iterative Refinement with Self-Feedback. In *Advances in Neural Information Processing Systems 36, NeurIPS 2023*.

OpenAI. 2024. Prompt Caching — OpenAI API Documentation. https://platform.openai.com/docs/guides/prompt-caching. Automatic prefix caching with discounted cached input tokens; see also https://openai.com/index/api-prompt-caching/; accessed July 2026.

Orvalho, P.; Janota, M.; and Manquinho, V. 2025. Counterexample Guided Program Repair Using Zero-Shot Learning and MaxSAT-based Fault Localization. In *Proceedings of the AAAI Conference on Artificial Intelligence*.

Penedo, G.; Kydlíček, H.; Allal, L. B.; Lozhkov, A.; Bakouch, E.; von Werra, L.; and Wolf, T. 2025. Codeforces. https://huggingface.co/datasets/open-r1/codeforces.

Rolim, R.; Soares, G.; D'Antoni, L.; Polozov, O.; Gulwani, S.; Gheyi, R.; Suzuki, R.; and Hartmann, B. 2017. Learning Syntactic Program Transformations from Examples. In *Proceedings of the 39th International Conference on Software Engineering (ICSE)*, 404–415.

Romera-Paredes, B.; Barekatain, M.; Novikov, A.; Balog, M.; Kumar, M. P.; Dupont, E.; Ruiz, F. J. R.; Ellenberg, J. S.; Wang, P.; Fawzi, O.; Kohli, P.; and Fawzi, A. 2024. Mathematical Discoveries from Program Search with Large Language Models. *Nature*, 625: 468–475.

Shinn, N.; Cassano, F.; Gopinath, A.; Narasimhan, K.; and Yao, S. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. In *Advances in Neural Information Processing Systems 36, NeurIPS 2023*.

Stengel-Eskin, E.; Prasad, A.; and Bansal, M. 2024. ReGAL: Refactoring Programs to Discover Generalizable Abstractions. In *Proceedings of the 41st International Conference on Machine Learning, ICML 2024*.

Wang, G.; Xie, Y.; Jiang, Y.; Mandlekar, A.; Xiao, C.; Zhu, Y.; Fan, L.; and Anandkumar, A. 2024. Voyager: An Open-Ended Embodied Agent with Large Language Models. *Transactions on Machine Learning Research*. ArXiv:2305.16291.

Wang, K.; Singh, R.; and Su, Z. 2018. Search, Align, and Repair: Data-Driven Feedback Generation for Introductory Programming Exercises. In *Proceedings of the 39th ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI 2018*. ACM.

Wang, Z.; Fried, D.; and Neubig, G. 2024. TroVE: Inducing Verifiable and Efficient Toolboxes for Solving Programmatic Tasks. In *Proceedings of the 41st International Conference on Machine Learning, ICML 2024*.

Xia, C. S.; and Zhang, L. 2022. Less training, more repairing please: revisiting automated program repair via zero-shot learning. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 959–971.

Xia, C. S.; and Zhang, L. 2024. Automated program repair via conversation: Fixing 162 out of 337 bugs for $0.42 each using chatgpt. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, 819–831.

Yang, B.; Tian, H.; Pian, W.; Yu, H.; Wang, H.; Klein, J.; Bissyandé, T. F.; and Jin, S. 2024. Cref: An llm-based conversational software repair framework for programming tutors. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, 882–894.

Yi, J.; Ahmed, U. Z.; Karkare, A.; Tan, S. H.; and Roychoudhury, A. 2017. A feasibility study of using automated program repair for introductory programming assignments. In *Proceedings of the 2017 11th joint meeting on foundations of software engineering*, 740–751.

Zhang, J.; Cambronero, J. P.; Gulwani, S.; Le, V.; Piskac, R.; Soares, G.; and Verbruggen, G. 2024. Pydex: Repairing bugs in introductory python assignments using llms. *Proceedings of the ACM on Programming Languages*, 8(OOPSLA1): 1100–1124.